Recursive Language Model For Poker

ARYAN ARORA, AARAV MULINTI, ARYAN JEENA, ADITHYA SRINIVASAN

PROBLEM

Long-Context Reasoning

As input length increases, attention diffuses across irrelevant tokens. Single-pass prompting fails on structured tasks like log parsing, multi-step retrieval, and aggregation

Technical Limitation

Irrelevant context wastes compute and increases hallucination risk, models lack iterative retrieval mechanisms

Core Challenge

Structured reasoning requires multi-step interaction with data, but standard LLM inference is a single forward pass with no external memory access.

OVERVIEW OF RLM

What RLMs do

- Train a language model to reason over long contexts by interacting with external memory via tool calls, rather than attending to everything at once

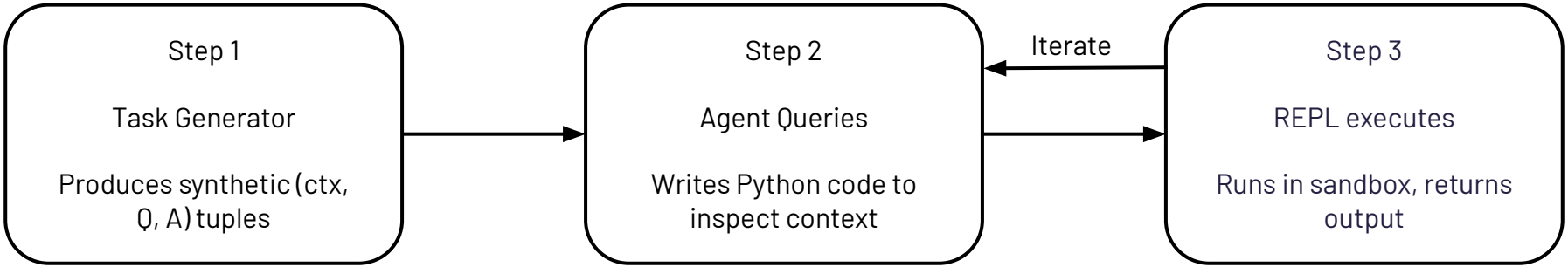
How they learn

- Start with behavior cloning; imitate an expert to get a reasonable base policy: $\mathscr{L}\mathrm{BC} = -\mathbb{E}[\log\pi\theta(a \mid s)]$
- Then improve with reinforcement learning — try actions, get rewards, update toward what worked: $\theta \leftarrow \theta + \eta, \widehat{\nabla J}$

Fundamentals

- Tasks sampled as (context, question, answer) triples; expectations estimated via Monte Carlo
- MDP: state = (context, query, history), action = code or final answer, trajectory $\tau = (s_1, a_1, \ldots, s_T, a_T)$
- Reward balances correctness, steps, and token usage: $R(\tau) = C - \lambda_s!\left(\frac{T}{T_{\max}}\right) - \lambda_t(N_{\text{tokens}})$
- Policy gradient, where b is an EMA baseline to reduce variance: $\nabla J(\theta) = \mathbb{E}!\left[\sum_t \nabla \log\pi_\theta(a_t \mid s_t) \cdot (R - b)\right]$

SYSTEM ARCHITECTURE



Long context lives in a Python variable - the agent interacts via tool calls, not prompts

WHY POKER

Structured but non-trivial decisions

Poker actions (fold/call/raise) are discrete and evaluable, but the reasoning behind them requires multi-step inference over hand strength, position, pot odds, and opponent modeling. It's not a lookup task.

Natural language + code duality

The agent must read a natural-language game state and produce executable Python, which is exactly the long-context structured reasoning your architecture is built around.

Rich teacher for BC

The heuristic (with opponent modeling, 3-step reasoning) gives high-quality BC labels and a clear 100% baseline to measure against, creating a well-defined gap to close.

Goal

- Model: Qwen2.5-Coder 1.5B + LoRA
- Make correct preflop decisions via code reasoning
- Metric: action-type match vs teacher
- Baseline: heuristic teacher = 100%
- Phases: BC (imitate) → RL (improve)

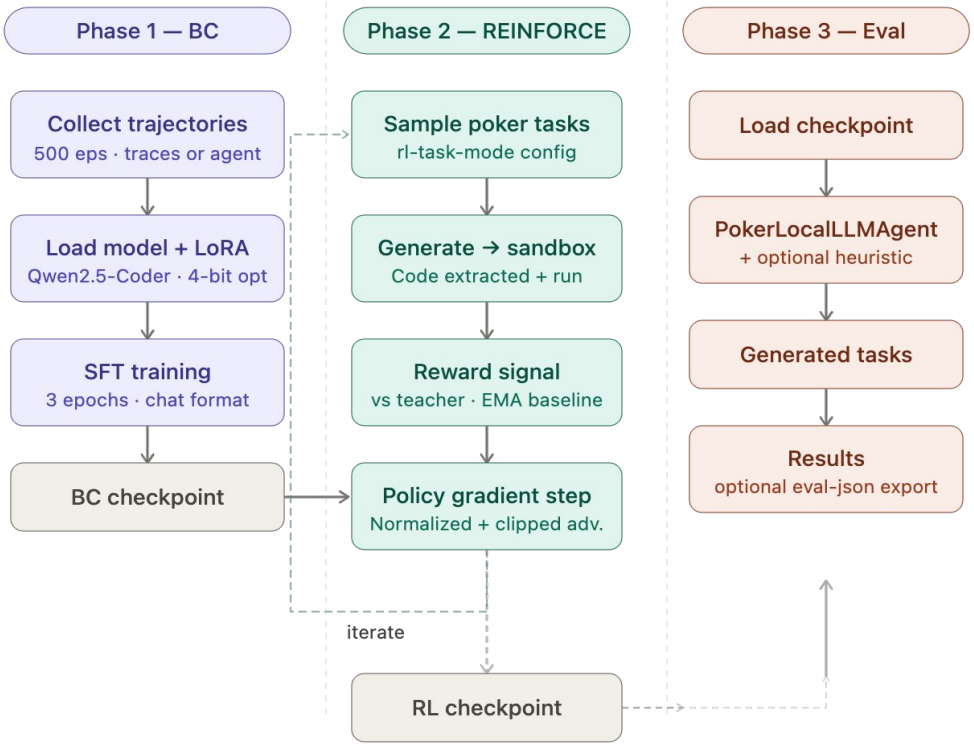
Data

- Inputs: hole cards, position, stack sizes, pot odds (~4,300 chars per example)
- 500 BC traces: (preflop state, heuristic action) pairs
- Teacher labels via opponent modeling + 3-step reasoning
- Difficulty bucketed: easy (heads-up, deep stacks) → hard (short stack, squeeze)

Env

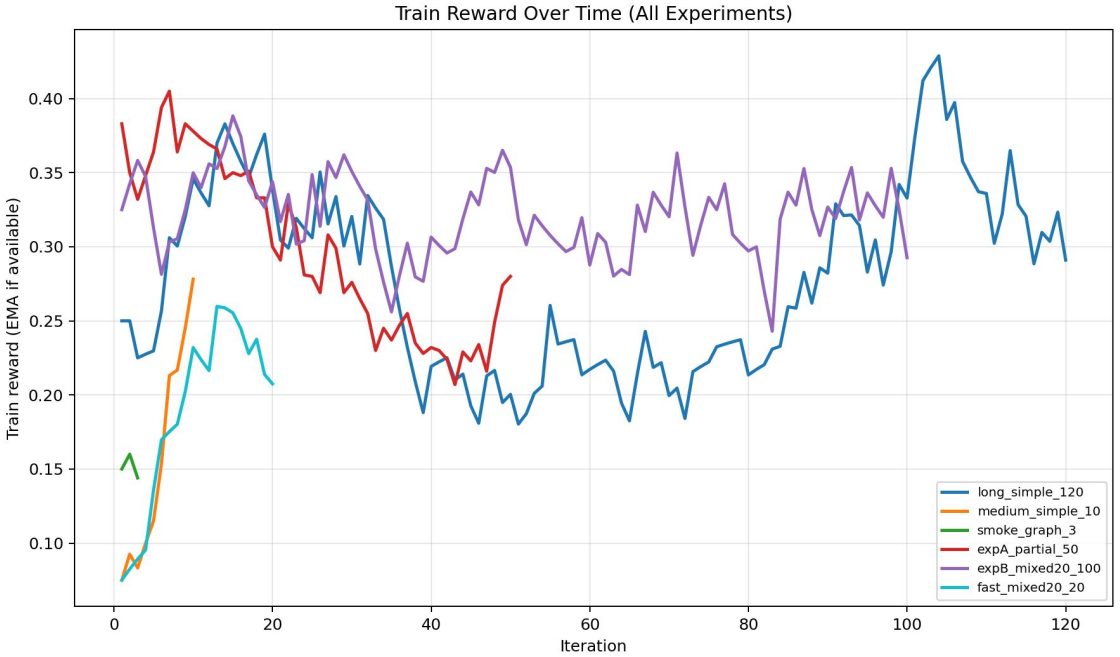
- Game state stored in a Python variable, not the prompt
- Actions: fold / call / raise
- Agent writes Python → REPL executes → result returned as next input
- Reward = action-type match vs teacher; no full EV calculation

TRAINING PIPELINE



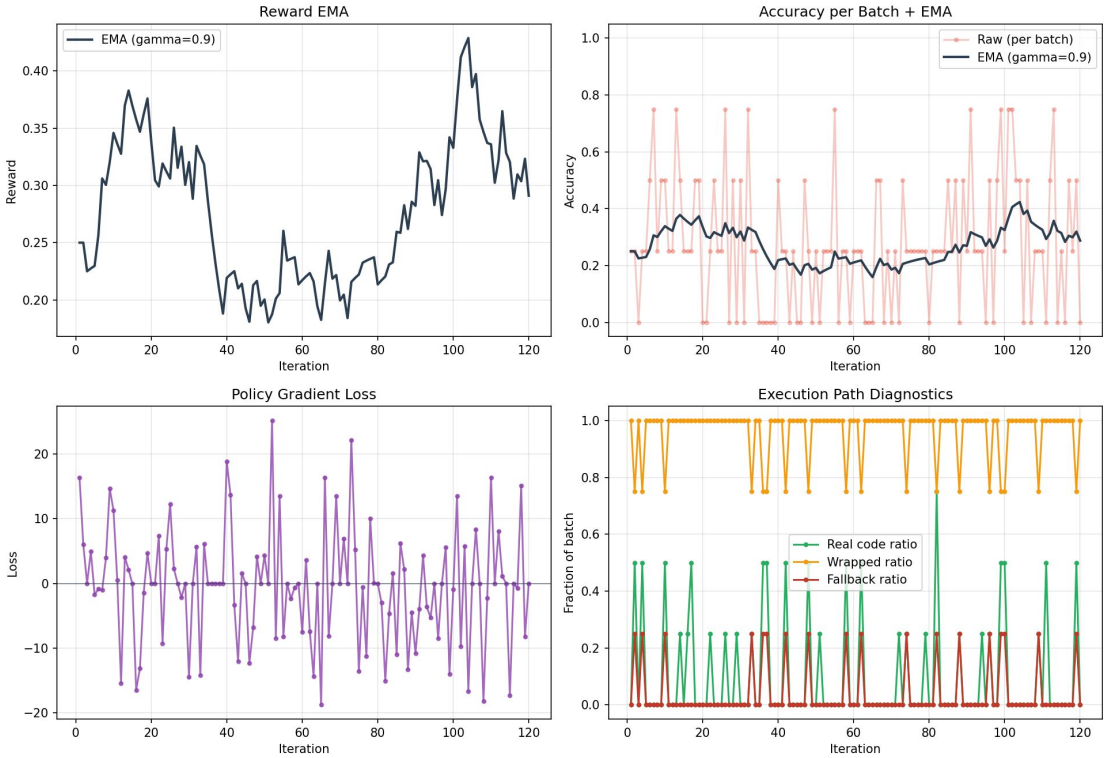
PROOF OF CONCEPT

- Started with short RL runs to verify the full BC -> RL pipeline worked.
- Confirmed LoRA + 4-bit fine-tuning, sandboxed Python execution, and REINFORCE updates all ran end-to-end.
- Early runs showed reward and training accuracy could move upward from near-zero levels.
- These runs established feasibility, but they did not include held-out evaluation.



LONGER TRAINING REVEALS INSTABILITY

- Scaled RL to a 120-iteration run to test whether more training improved the policy.
- Training reward and accuracy reached much higher peaks than in the short runs.
- However, performance later drifted downward and the final checkpoint was weaker than earlier peaks.
- This showed that longer RL training alone does not guarantee better performance.



CURRENT STATE VS. RLM

An RLM is defined by the tool loop, not just the model. So the model writes code, sandbox runs it, reads the results, and decides.

For example, Experiment B never closed the loop. Every iter: real_code = 0/4, wrapped = 4/4. The policy emitted one word ("fold") inside an empty code fence. The Python REPL we built might as well not exist.

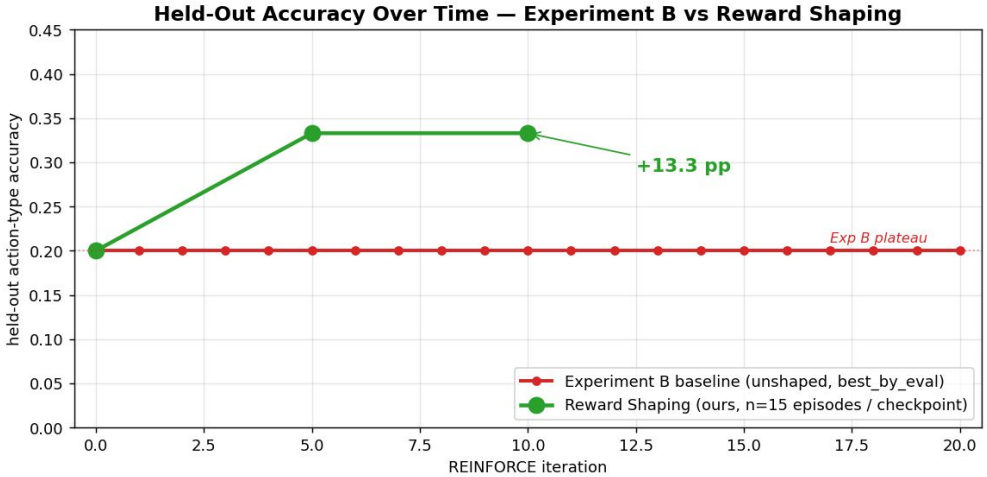
What comes next: the shaped-reward run is the first version (discussed next), is where we start rewarding real code, penalize the fallback, and the policy has to earn its accuracy through the sandbox and that's when our project turned to actually behaving like an RLM instead of a wrapper around a Qwen.

REWARD SHAPING

Experiment B hit a wall at 20% because the model gave up on code. When unsure, it just wrote one word ("fold") and collected whatever reward it could.

Our fix: reward the model for actually using Python (+0.3 for real code) and take points away when it falls back to one-word guesses (−0.2). Everything else about training is identical.

Result: 33.3% held-out +13 percentage points just from changing what we reward. Additionally, as you will see, with reward shaping the policy used code much more often.

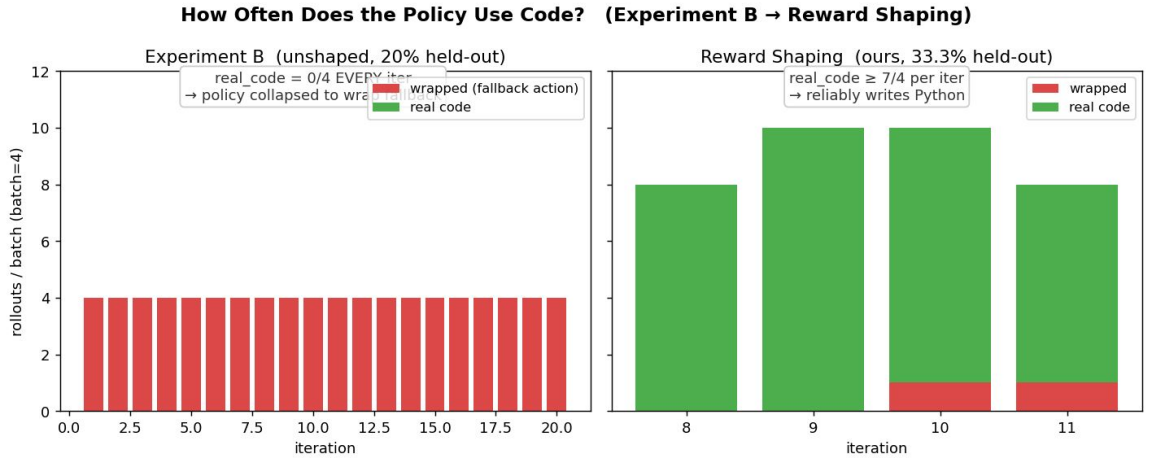


POLICY CODE USAGE

Before shaping, the model never wrote code — not once across 20 iterations. This includes for both Experiment B and for the Mixed Run.

After shaping, nearly every rollout is code.

But writing code isn't the same as writing code that runs — only ~5 out of 16 rollouts executed without errors. The natural next fix is to reward exec_ok, so the policy learns that code has to actually work, not just look like code.



EXAMPLE OF CODE WRITTEN

Three real samples from the shaped policy's training rollouts — exactly what the model wrote:

#1 → The easy shortcut — one-line "fold" inside a code fence earns the code-bonus without any reasoning:

```python
print("fold")
```

#2 → The model actually trying — scanning the game history for opponent stats:

```python
import re
lines = CONTEXT.split('\n')
for i, line in enumerate(lines):
    if '=== PREVIOUS HANDS' in line:
        history_start = i
        break
stats = {}
# (got truncated mid-loop, crashed on undefined
name)
```

#3 -> A real strategic decision — fold marginal hands against tight players, raise with flush draws:

```python
# Tight (low VPIP): respect bets,
fold marginals
elif opp_stats['VPIP'] <= 0.4:
    if strength['flush_draw']:
        print("call
{}".format(to_call))
    else:
        print("raise
{}".format(max(10, to_call * 1.5)))
```

LIMITATIONS

Proxy objective gap: our RL target is action-type agreement with a teacher, not direct long-horizon poker profitability

Teacher-bounded ceiling: supervision and reward are anchored to a heuristic teacher, so the model is optimized to mimic that policy family rather than discover fundamentally stronger play.

Sparse decision output space: evaluating fold/call/raise captures high-level action choice but under-represents finer strategy quality (e.g., sizing precision, exploitability).

Reproducibility uncertainty: current evidence shows capability but not yet stable, seed-robust performance guarantees.

TAKEAWAYS

RLM-style tool use is a strong fit for long-context, structured reasoning because the model can interact with external state and continuously improve upon self, instead of just a singular prompt.

Poker was a good test site; small action space, clear right answer, and there is true merit to reasoning behind an answer.

We had a lot of issues early as we missed something key – while our RL models were improving, they weren't actually using the REPL environment.

Next step — our reward-shaping fix got the model using the REPL again, but a lot of what it writes still crashes on bugs. The natural follow-up for us is to reward code that actually runs, not just any code, and potentially see how many next steps there are for reward hacking and optimizing our project!

REFERENCES

Zaremba et al. Recursive Language Models for Tool-Augmented Retrieval. 2025.
Davis, D. STAT 4830: Numerical Optimization for Machine Learning. §7 (RL for LMs), §9 (Benchmarking), §10 (Tuning Playbook). UPenn Spring 2026. https://damek.github.io/STAT-4830/
Williams, R. J. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. Machine Learning, 1992. (REINFORCE)
Ahmadian et al. Back to Basics: Revisiting REINFORCE Style Optimization for Learning from Human Feedback in LLMs. 2024.
Shao et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. 2024. (GRPO)
Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022.
Dettmers et al. QLoRA: Efficient Finetuning of Quantized LLMs. NeurIPS 2023. (4-bit NF4)
Dahl et al. Benchmarking Neural Network Training Algorithms. 2023. (AlgoPerf)
Shallue et al. Measuring the Effects of Data Parallelism on Neural Network Training. JMLR 2019.
Choi et al. On Empirical Comparisons of Optimizers for Deep Learning. 2019.
Qwen Team. Qwen2.5-Coder Technical Report. 2024. (base model: Qwen2.5-Coder-1.5B-Instruct)
Von Werra et al. TRL: Transformer Reinforcement Learning. Hugging Face. https://github.com/huggingface/trl

THANK YOU